

Convolutional Neural Networks

Overview

- **design and implement Convolutional Neural Networks (CNNs)**

Async

Neural Networks for Images

- complexity issue with processing images with a NN
 - for a 1024 x 1024 image and a NN with 100 neuron input layer: 1024×3 (RGB channels) $\times 100 \approx 300$ million parameters
- locality
 - many useful features are local
 - can be computed from small patches of the image
 - ex: is Mr. Poopybutthole in this picture?



- you would not generally process the image all at once, but you would scan zone by zone classifying each edged object

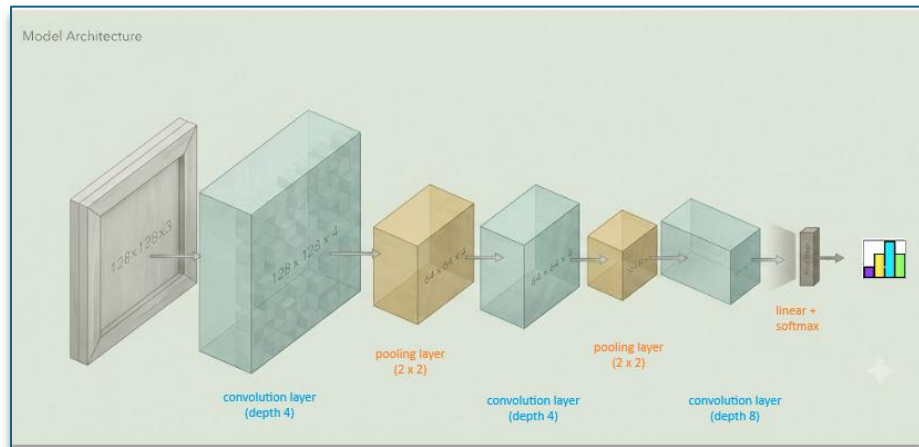


- until you identify the target object or run out of searchable objects / zones
- translation invariance
 - the exact coordinates of the feature in the image is irrelevant
- solution:
 - look at 3 x 3 patches of the image



	○ we still get a different output at each patch location ○ as abstract example: assuming each patch is 3×3 ▪ full image: $27 \times 18 \times 3 \times 100 \approx 146,000$ ▪ single patch: $3 \times 3 \times 3 \times 100 \approx 3,000$ ○ the features extracted from each patch is then combined to construct a feature map for the full image
Convolutions in Image Processing	• convolution ○ achieved through a “sliding dot product” ○ defined by weight matrix W_F (filter) ○ optionally an additive bias term
CNNs	• using the sliding window to calculate the dot product value for each pixel ○ the representation is still large (e.g., $1024 \times 1024 \times n_F$) ○ to compress, “pool” features form non-overlapping segments ○ example: take a max or average over each 2×2 region for each channel ▪ no learnable parameters here ▪ introduces spatial invariance ▪ produces: $512 \times 512 \times n_F$

- example CNN



Paddings and Edges

- stride
 - the number of spaces the sliding-window travels in a single sliding action
 - suppose an image is 7 x 7 and has a sliding window with no padding
 - a stride of one produces: 5 x 5
 - a stride of two produces: 3 x 3
 - we cannot apply a stride of 3 or more to this size matrix
- padding
 - add '0 pad border' around the image
 - this ensures that the output is the same size as the input (for stride = 1)
- output size formula

$$Output\ Size = \frac{X_{side} + 2P - f}{stride} + 1$$

	- pooling layer summary - input - tensor with dimensions $W_1 \times H_1 \times C$ - number of filters n_F - length of filter: F - length of strides: S - zero padding: P - output - tensor with dimensions $W_2 \times H_2 \times n_F$, where, $W_2 = \frac{W_1 - F + 2P}{S} + 1$ and $H_2 = \frac{H_1 - F + 2P}{S + 1}$ - number of learnable parameters: $n_F \times F \times F \times C$ weights and n_F bias terms
Examples of CNN Architectures	- LeNet for Handwritten Digit Recognition (1988: Yann LeCun!) - well-known 30+ year old handwriting reading tool - AlexNet (2012) - revolutionary model in terms of performance (16.4% error rate) - one of the real first deep learning successes - classic deep CNN architecture - first use of ReLU activations - trained with L2 regularization and dropout - VGG Net (2014) - large increase in depth over previous models - homogeneous stacks of smaller filters rather than layer-specific hyper-parameters 3 x 3 convolution filters with stride 1, padding 1 2 x 2 pooling filters with stride 2

- ResNet (2015)
 - 152 layers
 - no giant fully connected networks
 - all positions averaged (single layer)
 - skip and residual connections to provide better gradient optimization (enables training of really deep models)